

Endpoint Investigator — Documento Técnico

Avaliação Intermediária — Tecnologias Hackers · Insper · Prof. Rodolfo Avelino · Outubro de 2026

Dupla

- Guilherme Galvão Guedes — guilhermegg5@al.insper.edu.br
- Luiz Miguel Moraes Berredo — luizmmb1@al.insper.edu.br

1. Problema

Um analista que chega a um endpoint Linux suspeito costuma ter listas isoladas: o `ps` mostra processos, o `ls -l` mostra permissões, o `systemctl` mostra serviços. Cada lista, sozinha, diz pouco. Um serviço rodando como root é normal, um arquivo `0777` é uma má configuração mas não prova nada, e um processo com nome estranho pode ser legítimo. O que merece investigação costuma estar na **combinação**: por exemplo, um serviço privilegiado que executa um script que qualquer usuário pode alterar.

O Endpoint Investigator automatiza essa leitura combinada. Ele coleta um snapshot sob demanda do sistema, relaciona processos, permissões e serviços, e entrega achados que dizem o que foi observado, o que isso significa, o que é apenas hipótese e o que falta para confirmá-la. Não é um antivírus nem um EDR: o objetivo é apoiar a investigação, não decidir se houve incidente.

2. Estratégia de investigação

Seguimos o fluxo de referência do enunciado (coleta, normalização, correlação, evidências, hipóteses, resultado). Quatro princípios guiaram as decisões:

1. **Nenhuma evidência isolada vira achado.** "Serviço roda como root" ou "arquivo é `0777`" sozinhos não geram alerta. O raciocínio implementado é: serviço → executa como root → usa um script → o script pode ser alterado por usuário sem privilégio → possível relação de privilégio insegura. Isso não prova exploração.
2. **Permissões são analisadas por contexto.** Só inspecionamos arquivos que aparecem nos processos e serviços coletados (o executável e, quando ele é um interpretador como `bash` ou `python`, o script passado como argumento), nunca o filesystem inteiro.
3. **Evidência, interpretação, hipótese e evidência ausente são campos separados** de todo achado.
4. **Severidade e confiança são eixos independentes.** Severidade é quão grave seria se a hipótese fosse verdadeira; confiança é quão certos estamos de que ela é.

3. Arquitetura

```
COLETA → NORMALIZAÇÃO → CORRELAÇÃO → EVIDÊNCIAS/HIPÓTESES → RESULTADO
collectors/  normalizer/  correlator/  evidence/  reporter/
```

- **Coleta.** Um collector por dimensão (processos, permissões, serviços), cada um com duas implementações atrás da mesma interface: *real* (lê `/proc/<pid>/{status,stat,cmdline,exe}`, `os.stat()` e `systemctl`) e *dataset* (lê `processes.csv`, `permissions.csv` e `services.txt` do gerador fornecido). A mesma investigação roda, portanto, na VM Kali e sobre datasets reproduzíveis.
- **Normalização.** Os collectors produzem modelos comuns (`Process`, `FileResource`, `Service`, validados com pydantic). A classe `Snapshot` empacota uma coleta e constrói as relações: processo ↔ pai e filhos, serviço ↔ processo (pela linha de comando), serviço ↔ arquivos usados e arquivo ↔ permissão.
- **Correlação.** Funções de regra recebem o `Snapshot` e devolvem achados.
- **Evidências e hipóteses.** O modelo `Finding` carrega os quatro campos exigidos, mais severidade e confiança.
- **Resultado.** Relatório no terminal (biblioteca `rich`), com um painel por achado, colorido por severidade.

A execução é por CLI, sob demanda, sem agente permanente. Sem argumento, a ferramenta coleta o sistema real; com a pasta de um dataset, analisa o dataset.

4. Principais correlações

Implementamos duas correlações, o mínimo exigido.

C1 — Processo + Serviço + Permissão → hipótese de risco

(`servico_privilegiado_arquivo_gravavel`). Para cada serviço que roda como root, a ferramenta encontra o processo correspondente, os arquivos que ele usa e a permissão de cada um.

Condição	Severidade	Confiança
Arquivo gravável por qualquer usuário (<i>world-writable</i>)	alta	alta
Gravável só pelo grupo dono	média	baixa (não sabemos quem está no grupo)
Restrito ao dono	sem achado	—

C2 — Processo + PPID + Usuário → contexto de execução

(`processo_root_com_pai_nao_privilegiado`). Aponta processos root cujo pai roda como usuário comum.

Condição	Severidade	Confiança
<code>sudo</code> , <code>su</code> ou <code>pkexec</code> no processo ou no pai	baixa	alta (caminho esperado)
Sem mecanismo de elevação conhecido	alta	baixa (sabemos que aconteceu, não o motivo)

A queda normal de privilégio (`ssh` root abrindo o shell de um usuário) não dispara a C2, e um serviço root com script restrito (`0700`) não dispara a C1.

Exemplo de achado (C1, saída da ferramenta resumida):

- **Evidência:** o serviço `backup-agent.service` roda como root e executa `/opt/backup/backup.sh`, que tem permissão `0777` (dono root:root).
- **Interpretação:** o arquivo usado pelo serviço privilegiado pode ser alterado por qualquer usuário do sistema.
- **Hipótese:** se algum usuário alterar o arquivo, o conteúdo passa a rodar como root na próxima execução do serviço.
- **Evidência ausente:** nada confirma que o arquivo foi alterado, nem que o serviço executou logo depois de uma alteração suspeita. Auditoria de escrita (`auditd`) ou histórico de hash ajudariam.

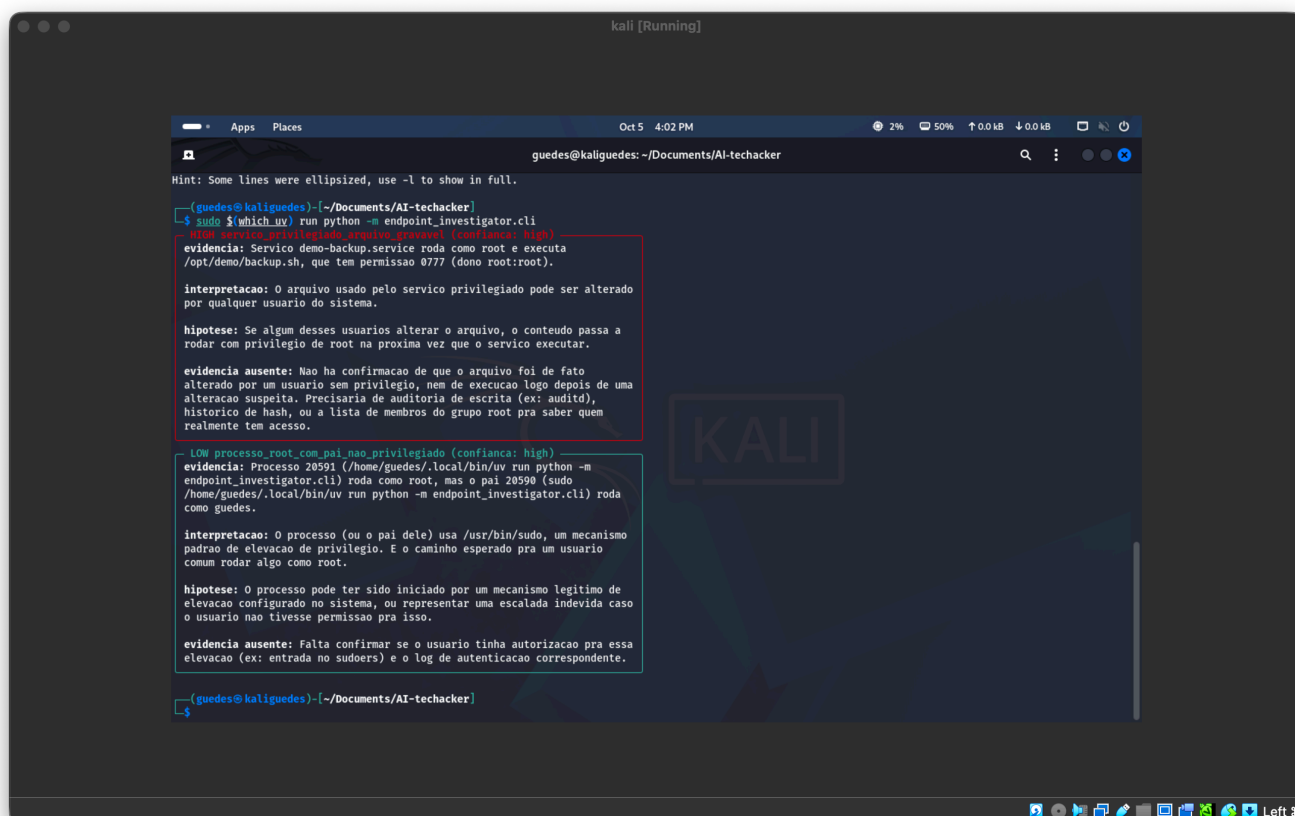
5. Decisões técnicas

- **Python 3.12 e uv, com poucas dependências:** `pydantic` (modelos) e `rich` (relatório). O `/proc` é lido direto, sem `psutil`, para trabalhar os conceitos de PID/PPID e UID/GID em vez de delegá-los a uma biblioteca.
- **Interface comum real/dataset** nos collectors, definida desde o início para não haver retrabalho e para permitir demonstrar tanto na VM quanto em datasets.
- **Permissões orientadas por contexto, garantidas em código:** o collector real levanta `ValueError` se for chamado sem a lista de caminhos de interesse.
- **Interpretadores:** quando o executável é `bash/sh/python`, o arquivo relevante é o script nos argumentos (`/bin/bash /opt/backup/backup.sh`). Sem isso, a C1 olharia só o `bash` e perderia o script.
- **Textos dos achados por templates determinísticos.** A evidência é montada com os dados da coleta; interpretação, hipótese e evidência ausente são textos fixos por regra, com poucas variáveis. Mesma entrada, mesma saída: é auditável e não depende de LLM.
- **Testabilidade sem Linux.** O desenvolvimento foi em macOS, então o `/proc` e o `systemctl` são injetáveis nos collectors reais, e o parsing é testado com dados fabricados (32 testes).
- **Resiliência.** Processos que somem durante a leitura do `/proc` e units que o `systemctl` não detalha são ignorados, sem derrubar a coleta.

6. Validação

- **Datasets do gerador.** Rodamos os seis cenários nomeados (`normal`, `permission`, `privileged_service`, `correlation`, `ambiguous`, `random`) e um lote de 20 datasets, todos sem erro. O `correlation` sempre gera o achado alta/alta; `normal`, `permission`, `privileged_service` e `ambiguous` não geram achados; no `random` a severidade acompanha o modo sorteado (`0777` → alta, `0770` → média, `0700` → nenhum achado).
- **Correções no gerador.** Achamos dois bugs no `generate_dataset.py` (o cenário `random_noise` quebrava ao ser sorteado, e `scenario_permission` aparecia como `normal` no `metadata.json`) e os corrigimos na nossa cópia.
- **VM Kali.** A execução em sistema real revelou dois problemas que os dados sintéticos não mostravam. Primeiro, um serviço (`auditd.service`) cujo `systemctl cat` falhava derrubava a coleta inteira. Segundo, o `sudo` real faz fork: o processo pai mantém o `sudo` e o filho já roda o comando final como root, então a C2 classificava a própria sessão da ferramenta como elevação desconhecida. Corrigimos os dois (a regra passou a olhar processo e pai) e, na nova execução, o `sudo` foi classificado corretamente (severidade baixa, confiança alta).

Para ver a C1 com dado real, criamos na VM um serviço de teste (**demo-backup.service**, root, executando um script **0777**). A ferramenta o apontou como achado alta/alta, como mostra o trecho da execução real abaixo:



```
Hint: Some lines were ellipsized, use -l to show in full.

(guedes@kaliguedes)-[~/Documents/AI-techacker]
$ sudo $(which uv) run python -m endpoint_investigator.cli
HIGH service_privilegiado_arquivo_gravavel (confianca: high)
evidencia: Servico demo-backup.service roda como root e executa
/opt/demo/backup.sh, que tem permissao 0777 (dono root:root).

interpretacao: O arquivo usado pelo servico privilegiado pode ser alterado
por qualquer usuario do sistema.

hipotese: Se algum desses usuarios alterar o arquivo, o conteudo passa a
rodar com privilegio de root na proxima vez que o servico executar.

evidencia ausente: Nao ha confirmacao de que o arquivo foi de fato
alterado por um usuario sem privilegio, nem de execucao logo depois de uma
alteracao suspeita. Precisaria de auditoria de escrita (ex: auditd),
historico de hash, ou a lista de membros do grupo root pra saber quem
realmente tem acesso.

LOW processo_root_com_pai_nao_privilegiado (confianca: high)
evidencia: Processo 20591 (/home/guedes/.local/bin/uv run python -m
endpoint_investigator.cli) roda como root, mas o pai 20590 (sudo
/home/guedes/.local/bin/uv run python -m endpoint_investigator.cli) roda
como guedes.

interpretacao: O processo (ou o pai dele) usa /usr/bin/sudo, um mecanismo
padrao de elevacao de privilegio. E o caminho esperado pra um usuario
comum rodar algo como root.

hipotese: O processo pode ter sido iniciado por um mecanismo legitimo de
elevacao configurado no sistema, ou representar uma escalada indevida caso
o usuario nao tivesse permissao pra isso.

evidencia ausente: Falta confirmar se o usuario tinha autorizacao pra essa
elevacao (ex: entrada no sudoers) e o log de autenticacao correspondente.

(guedes@kaliguedes)-[~/Documents/AI-techacker]
$
```

7. Uso de IA

Usamos ferramentas de IA generativa como apoio ao desenvolvimento: Correção de código e dos casos de testes, depuração (incluindo a análise dos bugs do gerador e das falhas que ocorreram na VM) e redação da documentação.

A solução em si não usa nenhum modelo de linguagem. Coleta, normalização, correlação e geração dos textos dos achados são determinísticas. Decidimos não incluir uma camada de IA nesta versão. Se for incluída no futuro, só poderá entrar depois da correlação, explicando achados já estruturados, nunca recebendo dados brutos para "analisar a máquina".

8. Limitações

- **Escopo.** Só processos, permissões e serviços: sem conexões de rede, logs, hashes, usuários/grupos ou persistência, e apenas duas das quatro correlações sugeridas. No cenário **ambiguous** (conexão externa feita por serviço root) a ferramenta não gera achado, mas não porque analisou a conexão: ela não coleta rede. É um ponto cego, não uma conclusão.
- **UID real, não efetivo.** Lemos o primeiro campo **Uid:** de **/proc/<pid>/status**, então binários setuid (que mudam só o UID efetivo) não são detectados pela C2.
- **Só o arquivo, não o diretório.** A C1 verifica a permissão do script, mas não a do diretório que o contém (um diretório gravável permitiria substituir o script mesmo com o arquivo restrito).
- **Associação serviço → processo por igualdade exata** entre o **ExecStart** (primeira linha do unit file, sem considerar drop-ins, variáveis ou prefixos como **–**) e a linha de comando do processo.

Quando não casa, o serviço fica sem processo associado e sem achados: um falso negativo silencioso.

- **Listas fixas** de interpretadores (`bash`, `sh`, `python`) e de ferramentas de elevação (`sudo`, `su`, `pkexec`). `doas`, `runuser` ou outros interpretadores (`perl`, `ruby`) escapam, e só caminhos absolutos nos argumentos são considerados.
- **Possíveis falsos positivos.** A C2 pode marcar elevações legítimas feitas por mecanismos fora da lista, e a C1 pode marcar arquivos graváveis só por grupos pequenos e confiáveis, por isso a confiança baixa nesse caso.
- **Sem achado explícito de "inconclusivo".** Quando falta a permissão de um recurso, a regra não gera achado; a incerteza aparece apenas nos campos de confiança e evidência ausente dos achados que existem.
- **Snapshot não atômico.** A leitura do `/proc` leva tempo, e processos podem surgir ou sumir durante a coleta.
- **Ambiente.** O modo real exige Linux com `systemd` e, para ver todos os processos, `root`. Foi validado numa VM Kali, não em outras distribuições.